

Conceitos preliminares

Introdução

Geometria Euclidiana

Objetivo: estudar construções geométricas.

Do ponto de vista algorítmico, existe um certo número de operações elementares, a serem executadas com régua e compasso, a partir das quais as construções desejadas deverão ser efetuadas.

Construções com régua e compasso são conhecidas como **construções elementares** na geometria euclidiana. Alguns exemplos:

- C1: dados dois pontos a e b , obter a reta definida por eles;
- C2: traçar um círculo de centro o e raio igual à medida do segmento ab ;
- C3: obter a interseção de duas retas, de dois círculos, ou de uma reta e um círculo.

Introdução

Geometria Euclidiana

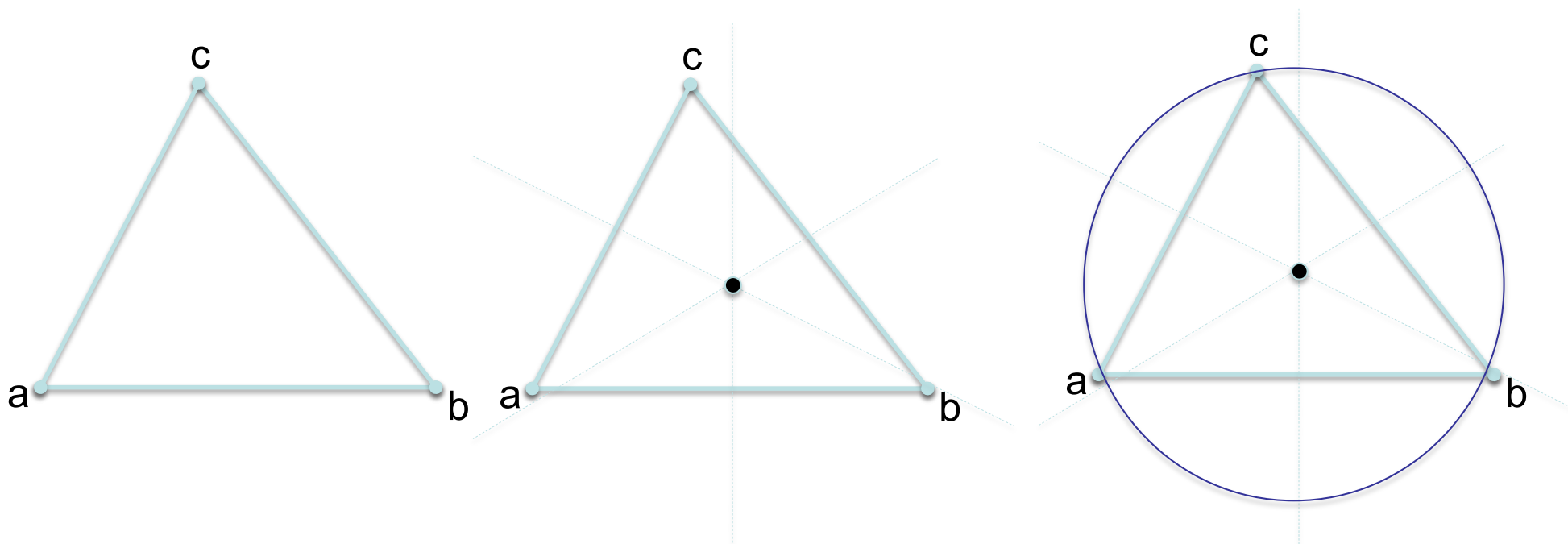
Exemplo: obter o círculo que contém três pontos não colineares a , b e c dados.

Introdução

Geometria Euclidiana

Exemplo: obter o círculo que contém três pontos não colineares a , b e c dados.

Solução (**usando régua e compasso**): obter o centro do círculo, que é a interseção das mediatrizes dos segmentos ab , ac e bc . Para obter tal ponto, podemos usar a seguinte sequência de construções elementares:



Introdução

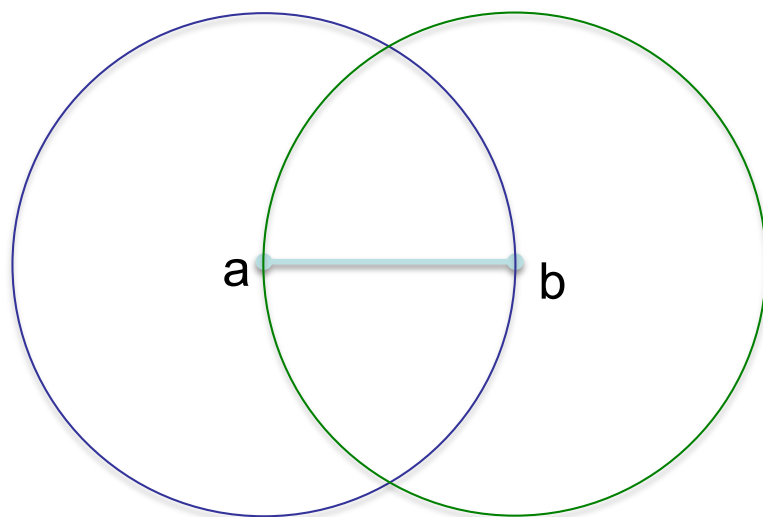
Geometria Euclidiana

Exemplo: obter o círculo que contém três pontos não colineares a , b e c dados.

Solução (**usando construções elementares**):

1) Aplicamos C2 para traçar, com centros em a e b , círculos de mesmo raio (por exemplo, de medida igual à do segmento ab);

C2: traçar um círculo de centro a (ou b) e raio igual à medida do segmento ab .



Introdução

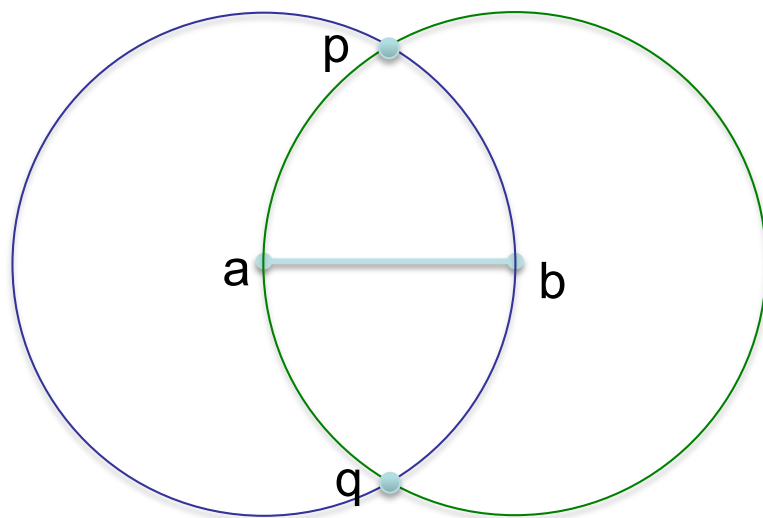
Geometria Euclidiana

Exemplo: obter o círculo que contém três pontos não colineares a , b e c dados.

Solução (**usando construções elementares**):

2) Usamos C3 para obter os pontos p e q de interseção destes círculos (a existência de pontos de interseção requer que o raio seja maior que a metade de ab , o que ocorre caso o raio seja tomado igual a ab);

C3: obter a interseção de duas retas, de dois círculos, ou de uma reta e um círculo.



Introdução

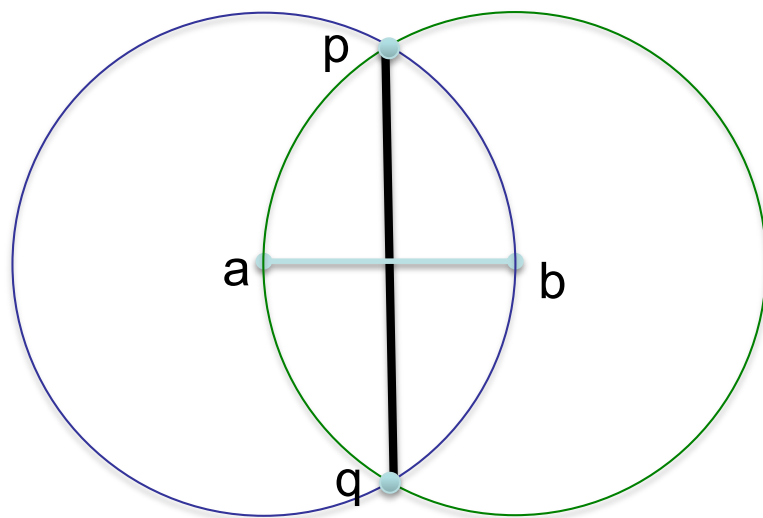
Geometria Euclidiana

Exemplo: obter o círculo que contém três pontos não colineares a , b e c dados.

Solução (**usando construções elementares**):

3) Usamos C1 para obter a reta definida por p e q . Essa reta é a mediatriz de ab ;

C1: dados dois pontos a e b , obter a reta definida por eles.



Introdução

Geometria Euclidiana

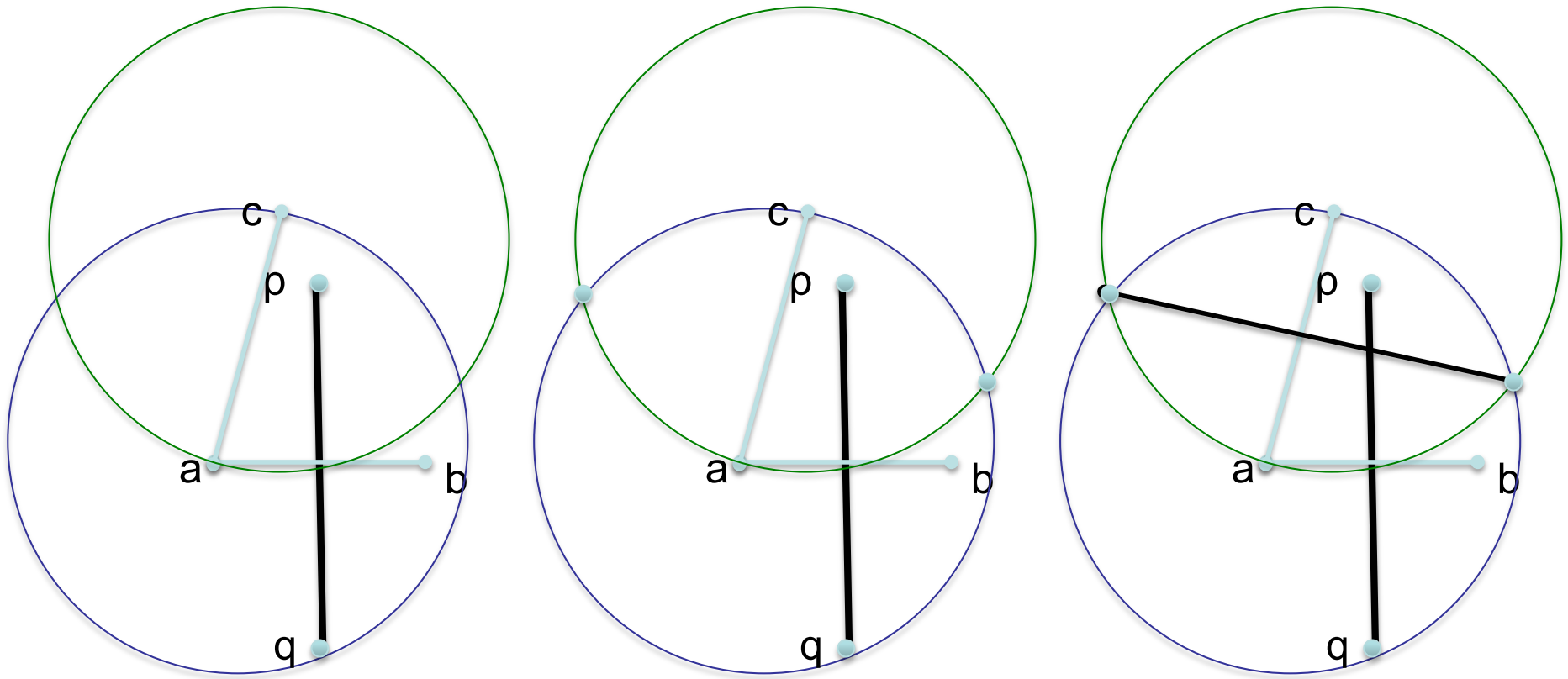
Exemplo: obter o círculo que contém três pontos não colineares a , b e c dados.

Solução (**usando construções elementares**):

4) Aplicamos a mesma sequência anterior para obter a mediatriz de ac ;

Introdução

Geometria Euclidiana



Introdução

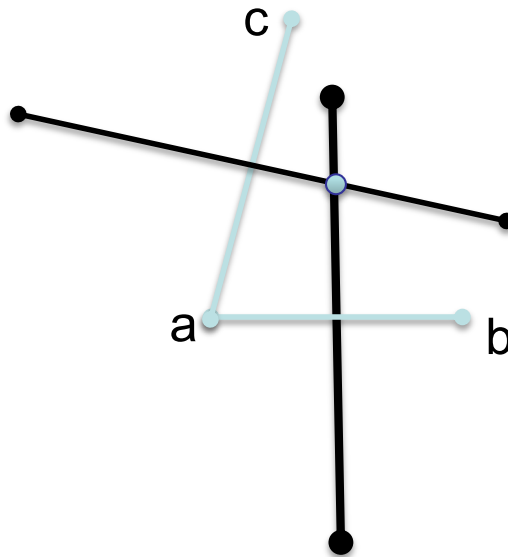
Geometria Euclidiana

Exemplo: obter o círculo que contém três pontos não colineares a , b e c dados.

Solução (**usando construções elementares**):

5) Usamos C3 para obter o centro o do círculo como a interseção das mediatrizes de ab e ac .

C3: obter a interseção de duas retas, de dois círculos, ou de uma reta e um círculo.



Introdução

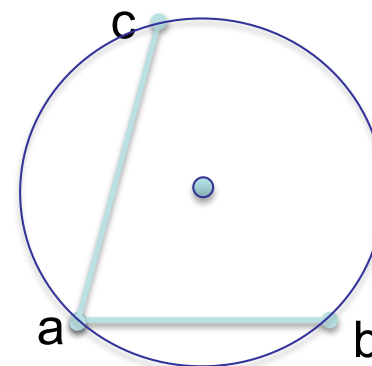
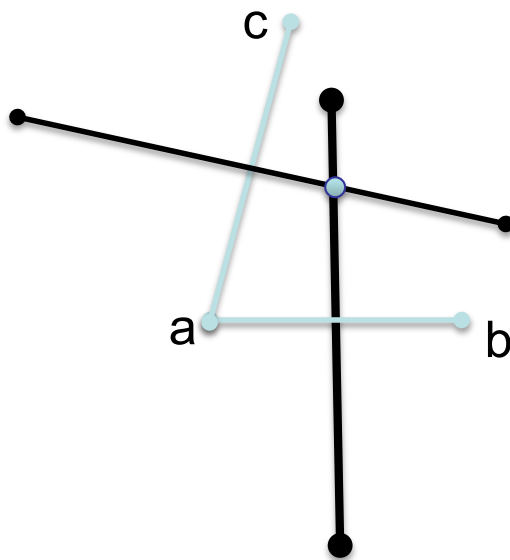
Geometria Euclidiana

Exemplo: obter o círculo que contém três pontos não colineares a , b e c dados.

Solução (**usando construções elementares**):

5) Usamos C3 para obter o centro o do círculo como a interseção das mediatrizes de ab e ac .

C3: obter a interseção de duas retas, de dois círculos, ou de uma reta e um círculo.



Introdução

Geometria Computacional

Estudar problemas geométricos, utilizando as construções elementares da geometria euclidiana, sob o ponto de vista da Análise de Complexidade de Algoritmos.

Análise de complexidade computacional usa o conceito de ***simplicidade*** de uma construção, definida pelo número de construções elementares que a compõem.

Atualmente, a análise do desempenho de algoritmos é realizada em função do tamanho do problema (isto é, da quantidade de dados que deverá ser processada pelo algoritmo).

Introdução

Geometria Computacional

De forma análoga ao estudo de construções geométricas euclidianas, a Geometria Computacional utiliza-se de algoritmos para a resolução de problemas geométricos. **A diferença é que esses problemas geométricos são representados por dados armazenados em computador.**

Portanto, para estabelecer os fundamentos teóricos da Geometria Computacional, é necessário escolher um **modelo computacional** que estabeleça como um problema geométrico pode ser representado em um computador e que fixe as operações elementares (ou **primitivas**) que irão operar sobre essas representações.

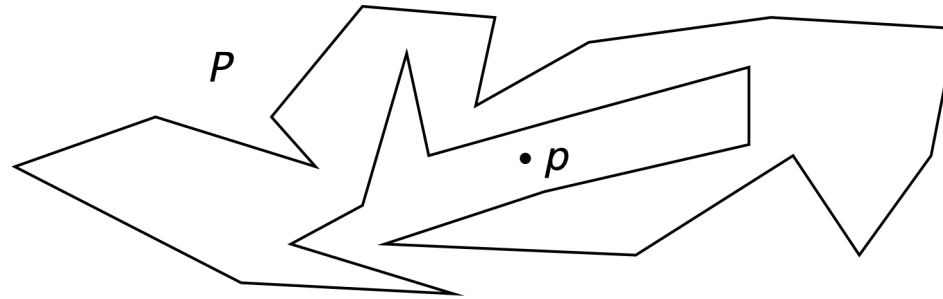
Modelos de Complexidade Computacional

Uma importante parte da Teoria da Computação tem por objetivo o estudo de modelos de computação, que procuram abstrair e simplificar o comportamento de computadores digitais.

- Modelo mais conhecido: **Máquina de Turing**, utilizada para estabelecer resultados básicos em computabilidade e complexidade computacional;
- O estudo desses modelos não é o objeto deste curso;
- Vamos utilizar informalmente um modelo computacional específico e enunciar os resultados de interesse para a Geometria Computacional.

Modelos de Complexidade Computacional

Conceito básico: Diferenciação entre problema, instância e algoritmo

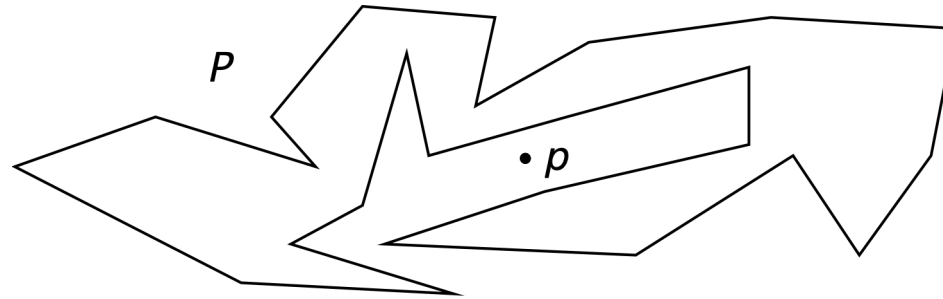


Problema: PONTO

EM POLÍGONO: Dado um polígono plano simples P e um ponto p do plano, decidir se p é interior ou não ao polígono P . Um polígono simples é um polígono em que os lados não se cruzam, a não ser nos vértices.

Modelos de Complexidade Computacional

Conceito básico: Diferenciação entre problema, instância e algoritmo

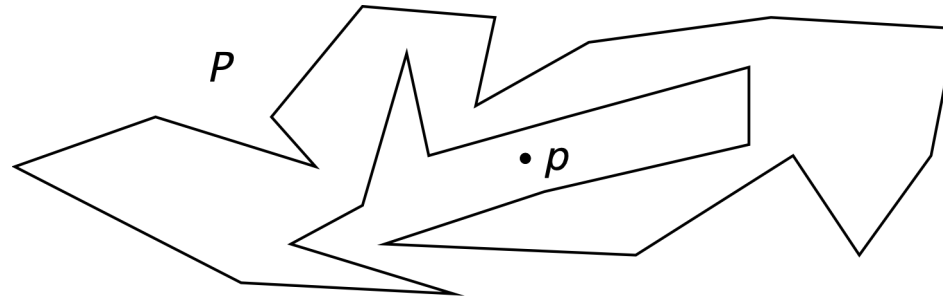


Problema: **PONTO EM POLÍGONO:** Dado um polígono plano simples P e um ponto p do plano, decidir se p é interior ou não ao polígono P . Um polígono simples é um polígono em que os lados não se cruzam, a não ser nos vértices.

Instância: Cada par (P, p) caracteriza uma instância específica deste problema.

Modelos de Complexidade Computacional

Conceito básico: Diferenciação entre problema, instância e algoritmo



Problema: **PONTO EM POLÍGONO:** Dado um polígono plano simples P e um ponto p do plano, decidir se p é interior ou não ao polígono P . Um polígono simples é um polígono em que os lados não se cruzam, a não ser nos vértices.

Instância: Cada par (P, p) caracteriza uma instância específica deste problema.

Algoritmo: Um algoritmo para PONTO EM POLÍGONO é uma lista de passos que, dada qualquer instância (P, p) , pára após um número finito de passos com a conclusão correta a respeito da inclusão de p no polígono P .

Modelos de Complexidade Computacional

Questões de interesse para este problema:

- Existe algum algoritmo que resolve o problema? (Isto exige que o algoritmo pare após um número finito de passos para qualquer instância do problema);
- Dado um certo algoritmo A para o problema, quão eficiente é este algoritmo?
- Dados dois algoritmos A e B , qual deles é superior?
- Dentre todos os algoritmos que resolvem o problema, qual deles é o melhor?

Modelos de Complexidade Computacional

Considerações para atender à existência de algoritmos:

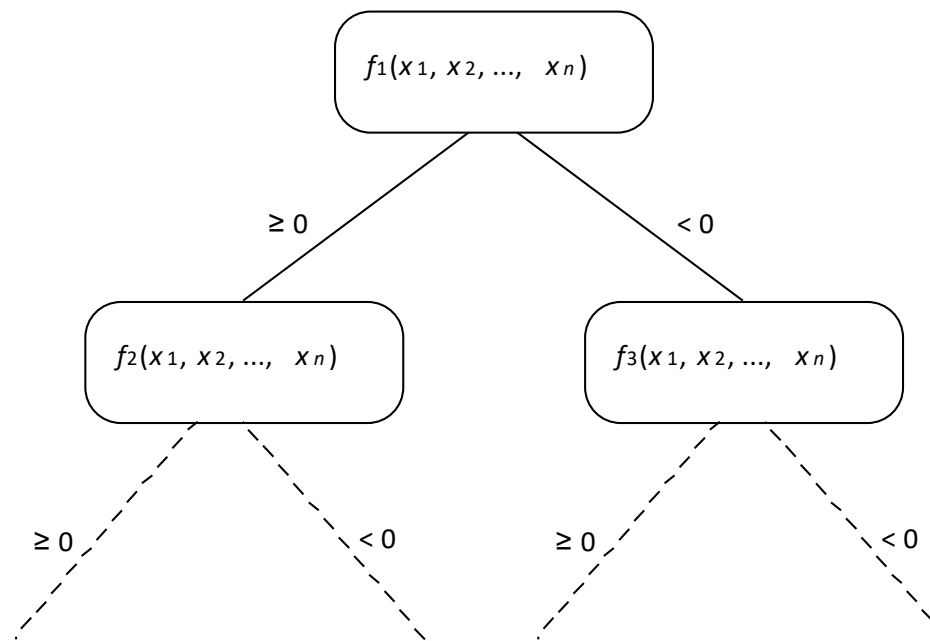
1) Estabelecer a natureza e a forma dos dados sobre os quais as instruções de tais algoritmos devem operar. Portanto:

- cada elemento de armazenamento de dados é capaz de armazenar um único número real;
- são considerados válidos quaisquer passos envolvendo operações aritméticas básicas (+, -, *, /), extração de raiz quadrada e comparações entre números reais.

Modelos de Complexidade Computacional

Modelo mais utilizado: **Árvores de decisões algébricas**

- Neste modelo, um algoritmo é representado por uma árvore em que cada nó representa um passo, onde uma certa função algébrica é avaliada. De acordo com o sinal do resultado, o algoritmo prossegue para um dos descendentes do nó em questão.



Modelos de Complexidade Computacional

Considerações para analisar a eficiência de algoritmos:

- 1) Alternativa 1: medir a eficiência (ou a complexidade) de um algoritmo pelo tempo necessário à sua execução, que por sua vez pode ser medido pelo número de passos elementares necessários ao seu término
 - Problema: é possível que dois algoritmos A e B sejam tais que A opera melhor sobre certas instâncias enquanto B opera melhor sobre outras.

Modelos de Complexidade Computacional

Considerações para analisar a eficiência de algoritmos:

2) Alternativa 2: exprimir a complexidade do algoritmo em função do tamanho da instância

- Problema: mesmo sobre instâncias de mesmo tamanho, o desempenho pode não ser uniforme.

Modelos de Complexidade Computacional

Considerações para analisar a eficiência de algoritmos:

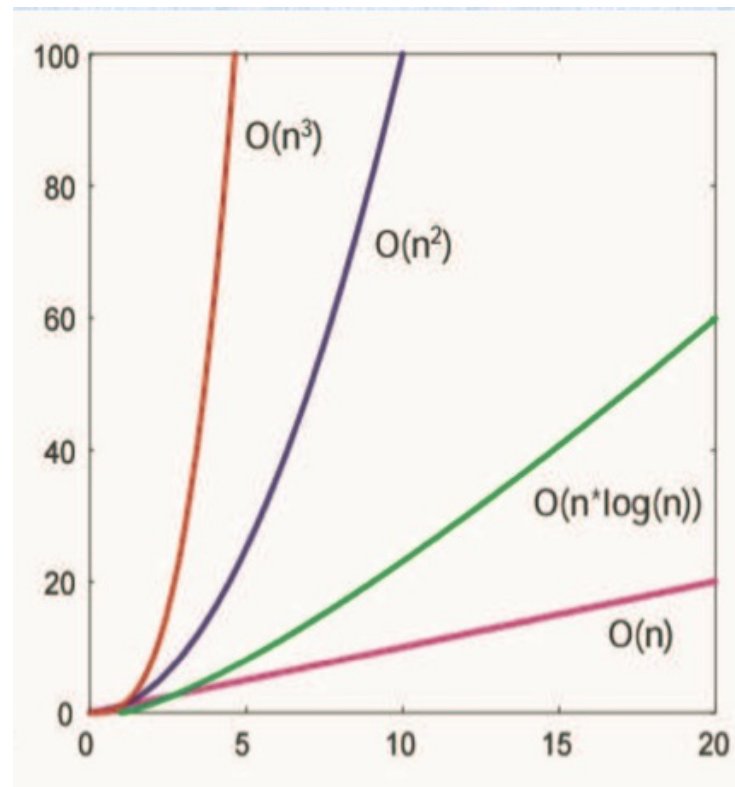
2) Alternativa 2: exprimir a complexidade do algoritmo em função do tamanho da instância

- Problema: mesmo sobre instâncias de mesmo tamanho, o desempenho pode não ser uniforme.
- Soluções:
 - A) Considerar um desempenho médio sobre todas as instâncias de um certo tamanho n (admitindo que as instâncias ocorrem segundo alguma lei de distribuição de probabilidades);
 - B) Considerar o caso mais desfavorável.

Modelos de Complexidade Computacional

Definição:

Um algoritmo A para resolver um problema P tem **complexidade** $O(f(n))$ se existe uma constante $k > 0$ e um natural N tais que, para qualquer instância de P de tamanho $n \gg N$, o número de passos de A necessários para resolver essa instância é, no máximo, $kf(n)$.



Modelos de Complexidade Computacional

A inclusão da constante multiplicativa k nesta definição tem algumas consequências, por exemplo:

- 1) Algoritmos de mesma complexidade teórica podem ter desempenhos bem diferentes, já que a constante multiplicativa embutida na complexidade pode ser arbitrariamente grande.
- 2) A análise tem caráter **assintótico**, já que o número de passos só precisa ser limitado por $kf(n)$ para valores de n superiores a um certo N . Na prática, pode ocorrer que os casos de interesse sejam exatamente aqueles em que n é menor que N .

Modelos de Complexidade Computacional

Conclusão:

Análise assintótica do caso mais desfavorável é a principal técnica de análise de algoritmos e fornece informação a respeito de como o tempo necessário para a execução do algoritmo cresce com o tamanho da instância.

Um “**bom**” algoritmo é aquele no qual tal crescimento não seja muito rápido.

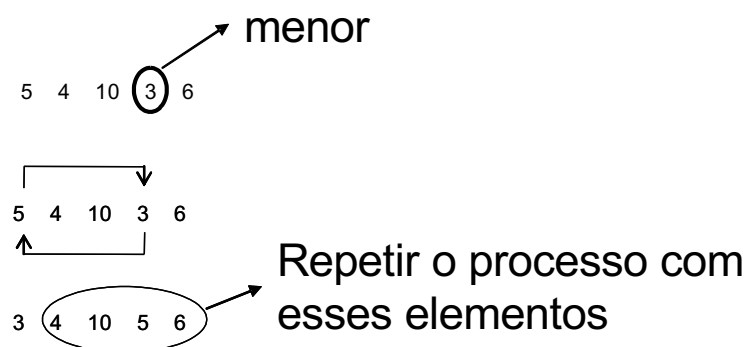
Exemplo: Algoritmos de Ordenação

Problema: Dados n números reais $x_1, x_2, x_3, \dots, x_n$, colocá-los em ordem crescente.

Exemplo: Algoritmos de Ordenação

Problema: Dados n números reais $x_1, x_2, x_3, \dots, x_n$, colocá-los em ordem crescente.

1º Algoritmo: Ordenação por seleção



Algoritmo

```
para  $i = 1..n-1$   
     $m = i$   
    para  $j = i+1..n$   
        se  $x_j < x_m$  então  $m = j$   
    troque  $x_i$  com  $x_m$ 
```

Exemplo: Algoritmos de Ordenação

Análise:

- O tamanho de uma instância é dada pelo número n de reais a serem ordenados.
- Na primeira execução do corpo do loop principal, são feitas $n-1$ comparações e 1 troca, num total de n operações.
- Na segunda execução, $n-2$ comparações e 1 troca, e assim por diante.
- Logo, o número de passos requerido pelo algoritmo é:

$$n + (n-1) + (n-2) + \dots + 1 = \frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}$$

- Complexidade: ?

Exemplo: Algoritmos de Ordenação

Análise:

- O tamanho de uma instância é dada pelo número n de reais a serem ordenados.
- Na primeira execução do corpo do loop principal, são feitas $n-1$ comparações e 1 troca, num total de n operações.
- Na segunda execução, $n-2$ comparações e 1 troca, e assim por diante.
- Logo, o número de passos requerido pelo algoritmo é:

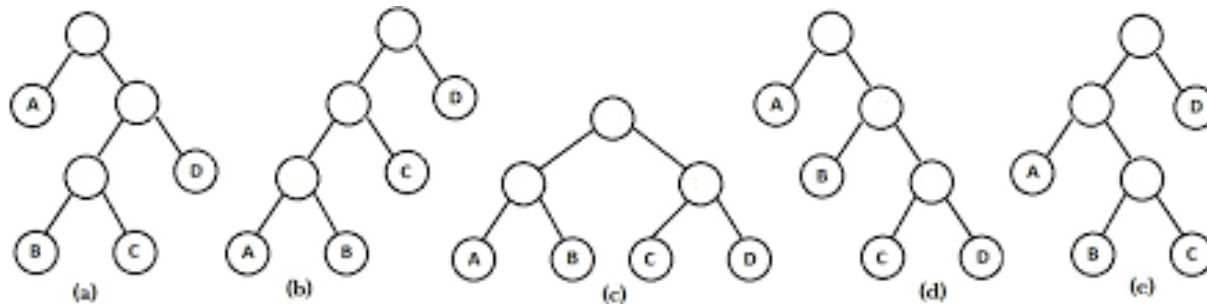
$$n + (n-1) + (n-2) + \dots + 1 = \frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}$$

- Complexidade: **$O(n^2)$**

Exemplo: Algoritmos de Ordenação

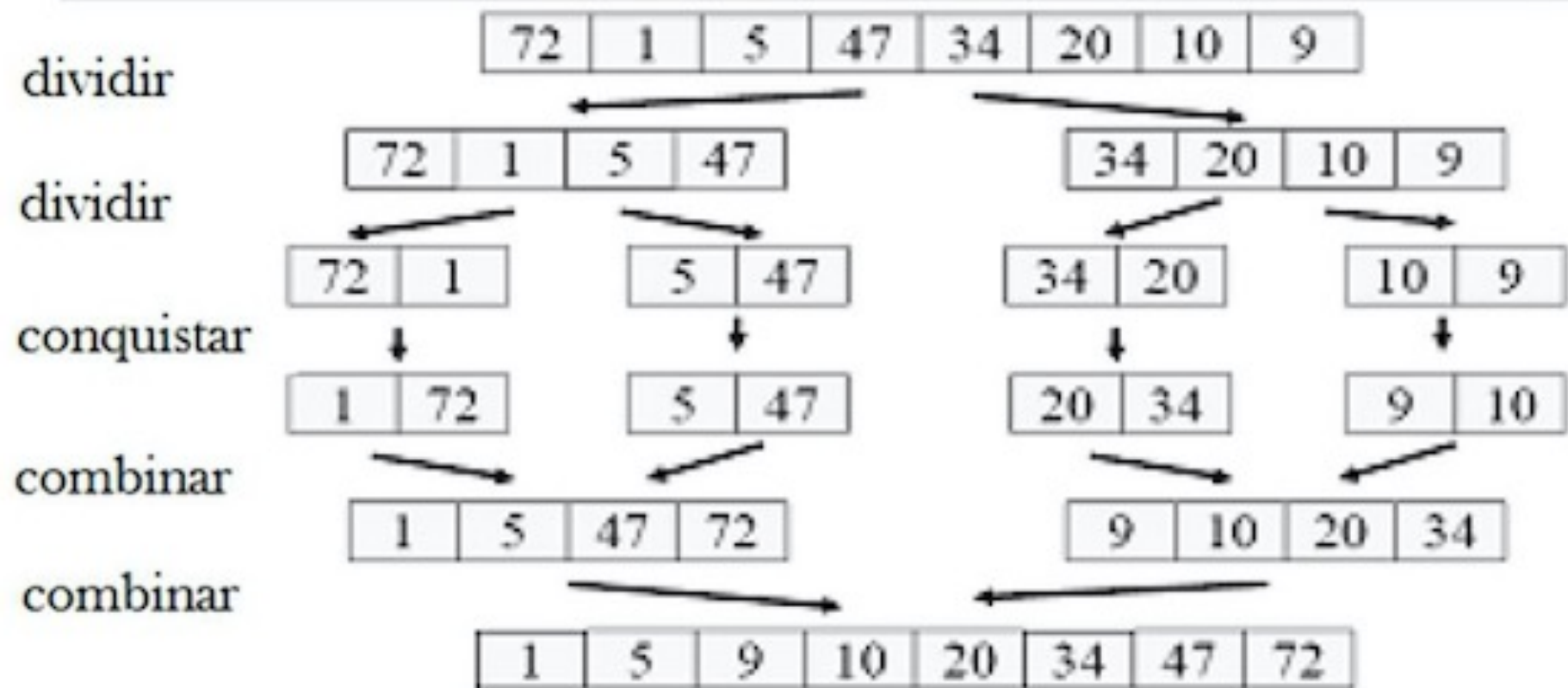
2º Algoritmo: Mergesort

- Este algoritmo usa um dos paradigmas fundamentais de construção de algoritmos, onde a solução de um problema é obtida através da resolução de problemas de tamanho menor. Esse paradigma é conhecido como ***dividir-para-conquistar*** e é o segredo da eficiência de muitos algoritmos.



- No Mergesort, ao invés de ordenar o conjunto de n números, dividimos esse conjunto em dois subconjuntos com $n/2$ elementos cada, ordenamos cada um desses conjuntos e reunimos esses subconjuntos já ordenados. Para ordenar cada um dos subconjuntos com $n/2$ elementos, aplicamos recursivamente o mesmo algoritmo.

Exemplo: Algoritmos de Ordenação



Exemplo: Algoritmos de Ordenação

```
mergesort(x,n)
    se  $n < 2$  então retorne           {caso básico da recursão}
     $m = n/2$ 
     $l = x[1..m]$                        {separação}
     $r = x[m+1..n]$ 
    mergesort(l,m)                     {recursão}
    mergesort(r,n-m)
     $i = 1$ 
     $j = 1$ 
    para  $k = 1..n$                        {combinação}
        se  $l_i < r_j$  então
             $x_k = l_i; i = i+1;$ 
        senão
             $x_k = r_j; j = j+1;$ 
```

Obs.: assumindo que existem elementos auxiliares (chamados **sentinelas**) tais que $l_{m+1} = +\infty = r_{n-m+1}$.

Exemplo: Algoritmos de Ordenação

Análise:

- Suponhamos que n é da forma $n = 2^p$ (ou seja, $p = \log n$), onde p é um inteiro positivo.
- Seja $T(n)$ a complexidade do algoritmo, expressa pelo número de passos necessários à sua execução.
- O algoritmo consiste em duas chamadas recursivas do mesmo algoritmo para instâncias de tamanho $n/2$, seguidas por uma etapa em que, a partir de dois conjuntos já ordenados, deve-se obter sua união, também ordenada.
- Essa união é obtida em um número de passos que é proporcional a n . Logo, a complexidade $T(n)$ é dada por:

$$T(n) = 2T(n/2) + kn$$

Exemplo: Algoritmos de Ordenação

Análise:

- Suponhamos que n é da forma $n = 2^p$ (ou seja, $p = \log n$), onde p é um inteiro positivo.
- Seja $T(n)$ a complexidade do algoritmo, expressa pelo número de passos necessários à sua execução.
- O algoritmo consiste em duas chamadas recursivas do mesmo algoritmo para instâncias de tamanho $n/2$, seguidas por uma etapa em que, a partir de dois conjuntos já ordenados, deve-se obter sua união, também ordenada.
- Essa união é obtida em um número de passos que é proporcional a n . Logo, a complexidade $T(n)$ é dada por:

$$T(n) = 2T(n/2) + kn$$

$$T(n/2) = 2T(n/4) + kn/2 \longrightarrow T(n) = 4T(n/4) + 2kn$$

$$T(n/4) = 2T(n/8) + kn/4 \longrightarrow T(n) = 8T(n/8) + 3kn$$

$$T(n) = 2^p T(n/2^p) + pkn = nT(1) + k(\log n)n$$

Complexidade:
???

Exemplo: Algoritmos de Ordenação

Análise:

- Suponhamos que n é da forma $n = 2^p$ (ou seja, $p = \log n$), onde p é um inteiro positivo.
- Seja $T(n)$ a complexidade do algoritmo, expressa pelo número de passos necessários à sua execução.
- O algoritmo consiste em duas chamadas recursivas do mesmo algoritmo para instâncias de tamanho $n/2$, seguidas por uma etapa em que, a partir de dois conjuntos já ordenados, deve-se obter sua união, também ordenada.
- Essa união é obtida em um número de passos que é proporcional a n . Logo, a complexidade $T(n)$ é dada por:

$$T(n) = 2T(n/2) + kn$$

$$T(n/2) = 2T(n/4) + kn/2 \longrightarrow T(n) = 4T(n/4) + 2kn$$

$$T(n/4) = 2T(n/8) + kn/4 \longrightarrow T(n) = 8T(n/8) + 3kn$$

$$T(n) = 2^p T(n/2^p) + pkn = nT(1) + k(\log n)n$$

Complexidade:
 $T(n) = O(n \log n)$

Exemplo: Algoritmos de Ordenação

Análise:

- Como o crescimento da sequência $n \log n$ é mais lento que o de n^2 , este algoritmo é superior ao anterior do ponto de vista assintótico.
- Esta análise é característica de algoritmos recursivos do tipo dividir-para-conquistar, isto é, de algoritmos que possuem a seguinte estrutura:

Algoritmo Dividir-para-Conquistar:

- **Separação:** decompor a instância do problema em duas instâncias de tamanhos menores;
- **Recursão:** aplicar recursivamente o mesmo algoritmo para cada uma destas instâncias;
- **Combinação:** combinar os resultados para obter a solução da instância original.

Exemplo: Algoritmos de Ordenação

Desvantagem: faz uso de um array auxiliar, gerando um gasto extra de memória (o dobro).

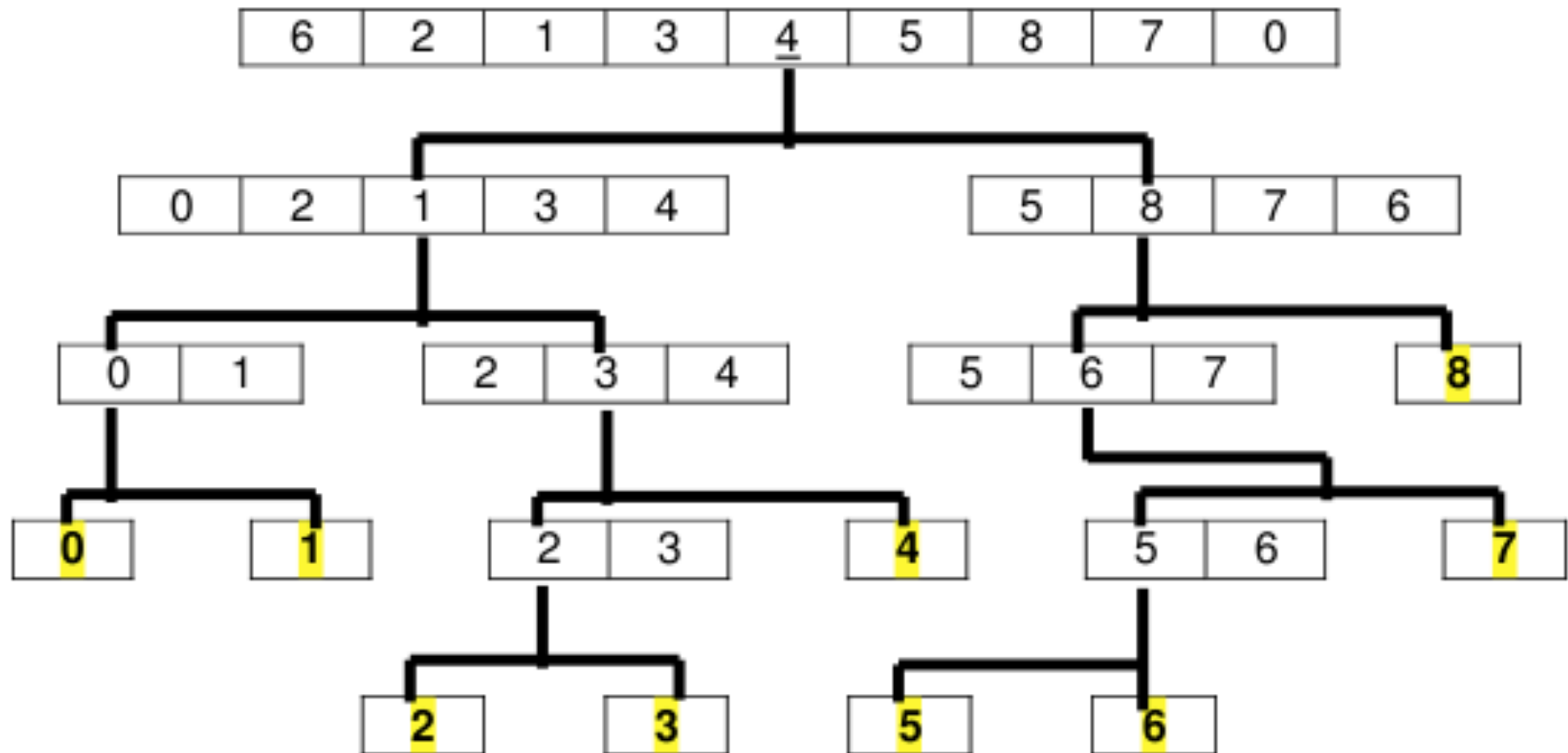
Teorema: Se um algoritmo recursivo do tipo *dividir-para-conquistar* para resolver um problema decompõe uma instância de tamanho n em duas instâncias de tamanho $n/2$ do mesmo problema, e se o processamento necessário à execução das etapas de *Separação* e *Combinação* tem, no total, complexidade $O(n)$, então o algoritmo resultante tem complexidade $O(n \log n)$.

Exemplo: Algoritmos de Ordenação

3º Algoritmo: Quicksort

- Também utiliza o paradigma de *dividir-para-conquistar*;
- Seu nome é indicativo do bom desempenho alcançado pelo algoritmo na prática;
- Ideia: toma-se um elemento do conjunto (por exemplo, x_1) e a etapa de **Separação** consiste em dividir o conjunto a ser ordenado em dois subconjuntos: o primeiro é composto por todos os elementos menores que x_1 e o segundo por todos os elementos maiores ou iguais a x_1 ;
- Cada um desses conjuntos é ordenado utilizando recursivamente o mesmo algoritmo. A etapa de **Combinação** é completamente trivial, já que a sequência ordenada consiste em listar os elementos (já ordenados) que são menores que x_1 , seguidos pelos elementos (também já ordenados) que são maiores ou iguais a x_1 .

Exemplo: Algoritmos de Ordenação



Exemplo: Algoritmos de Ordenação

```
quicksort(r,s)
    se  $s \leq r$  então retorne {caso básico}
     $v = x_r$ 
     $i = r$ 
     $j = s+1$ 
    repita {separação}
        repita  $i = i+1$  até  $x_i \geq v$ 
        repita  $j = j-1$  até  $x_j \leq v$ 
        troque  $x_i$  com  $x_j$ 
    até  $j \leq i$ 
    troque  $x_i$  com  $x_j$ 
    troque  $x_i$  com  $x_s$ 
    quicksort(r,i-1) {recursão}
    quicksort(i+1,s)
```

Exemplo: Algoritmos de Ordenação

Análise:

- O bom desempenho deste algoritmo na prática se deve ao fato de que, caso os elementos a serem ordenados sejam escolhidos aleatoriamente em um certo conjunto, o processo de separação tende a dividir o conjunto dado em dois conjuntos com aproximadamente o mesmo número de elementos. Como o processamento da separação requer $O(n)$ passos, uma análise semelhante à desenvolvida anteriormente permite demonstrar que a complexidade **média** do algoritmo é $O(n \log n)$ (sob a suposição de que os elementos a serem ordenados são independentemente escolhidos segundo alguma distribuição de probabilidade).
- No entanto, a complexidade deste algoritmo no pior caso é $O(n^2)$. Para se constatar este fato, basta observar que, caso o conjunto já esteja ordenado, o algoritmo descrito acima vai executar $n-1$ separações. A primeira exige $n-1$ comparações, a segunda $n-2$, e assim por diante, num total de $O(n^2)$ comparações.

Exemplo: Algoritmos de Ordenação

4º Algoritmo: Ordenação por inserção

- Usado em problemas de ordenação na qual desejamos obter soluções parciais. Mais precisamente, queremos ordenar um conjunto $x_1, x_2, \dots, x_n$ de números reais, examinando os elementos x_i um de cada vez, de tal forma que, após examinar x_i , já tenhamos ordenado $x_1, x_2, \dots, x_i$. Algoritmos deste tipo de são chamados **incrementais** ou **on-line**.

Algoritmo

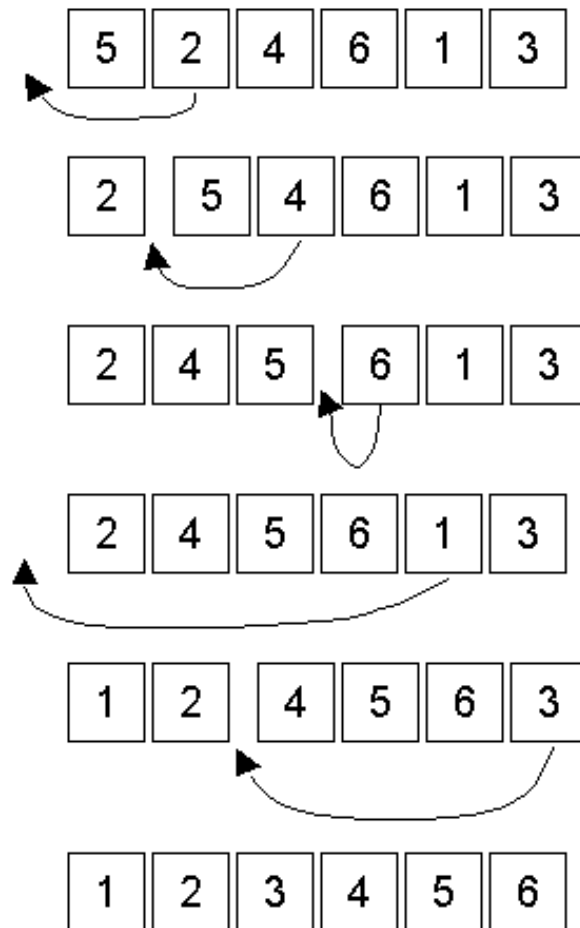
```
para  $i = 2..n-1$   
     $v = x_i$   
     $j = i$   
    enquanto  $x_{j-1} > v$   
         $x_j = x_{j-1}$   
         $j = j - 1$   
  
     $x_j = v$ 
```

(Assumindo sentinela $x_0 \leq x_i$.)

Exemplo: Algoritmos de Ordenação

- **Analogia:** O algoritmo de ordenação por inserção é bastante simples e eficiente para uma quantidade pequena de números. A idéia de funcionamento é semelhante ao ordenamento de cartas de baralho na mão de um jogador. A mão esquerda começa "vazia" e a mão direita insere uma "carta" de cada vez na posição correta. Ao final, quando todas as "cartas" foram inseridas, elas já estão ordenadas. Para encontrar, durante a inserção, a posição correta para um valor, compara-se este valor um-a-um com as cartas já na mão esquerda, até encontrarmos a posição correta.

Exemplo: Algoritmos de Ordenação



Exemplo: Algoritmos de Ordenação

- **Análise:** Neste algoritmo não há necessidade de utilização de nenhuma estrutura de dados extra, além do próprio vetor inicial com os valores. Considere que os números a serem ordenados estão armazenados num vetor. O algoritmo de ordenação por inserção tem complexidade proporcional a n^2 no pior caso, onde n é o número de elementos a serem ordenados.

Complexidade:
 $O(n^2)$

Exemplo: Algoritmos de Ordenação

Conclusão:

- Existem estruturas de dados capazes de realizar pesquisas e inserções em tempo logarítmico. Essas estruturas de dados são, em geral, árvores binárias modificadas de modo a garantir equilíbrio.
- Com essas estruturas, é possível implementar algoritmos incrementais ótimos para ordenação. As árvores balanceadas resolvem o problema de ordenação dinâmica de maneira ótima, pois são capazes de realizar inserções, remoções e pesquisas em tempo $O(\log n)$.
- No entanto, a implementação destas árvores balanceadas é delicada e foge do nosso escopo.
- Quais conclusões podemos chegar em relação ao Problema ORDENAÇÃO?